

Regional Surveillance Insights



A Django + MySQL web application that loads a regional public-health surveillance dataset (20 countries, 2021–2025), profiles and cleans it, and presents summary indicators, a data table, and a data-driven view of which countries need increased support ahead of the next biennium.

What it does

- **ETL** — reads 8 CSV files, profiles them, flags data-quality problems and loads them into MySQL (one reference table + per-country/year fact tables).
- **Dashboard** — regional summary cards, four charts, and the recommendations.
- **Countries needing support** — every country ranked by a transparent **Support-Need Index (SNI)**.
- **Data table** — the full country-year dataset with year/country filters, free-text search and click-to-sort.

Tech stack

Layer	Choice	Why
Database	MySQL 8.4	Relational data with clear keys; mature, well-understood, easy to operate.
Backend	Django 5.1	Batteries-included: ORM, migrations, admin, management commands for the ETL.
ETL	pandas	Fast profiling and CSV handling; run as a Django management command so it shares the models.
Charts	Chart.js	Lightweight, no build step.
Packaging	Docker Compose	One command brings up MySQL + the app, fully reproducible.

Quick start (Docker — recommended)

```
cp .env.example .env
docker compose up --build
```

Then open <http://localhost:8000>. On first start the web container waits for MySQL, runs migrations, runs the ETL (`load_data`) and serves the app.

Useful commands:

```
docker compose up --build -d          # run in the background
docker compose logs -f web            # follow the app logs
docker compose down                  # stop (keep the data)
docker compose down -v               # stop and wipe the database (fresh load next time)
docker compose exec -e MYSQL_USER=root -e MYSQL_PASSWORD=rootpw web \
    python manage.py test surveillance # run the test suite
```

Local run (without Docker)

Requires **Python 3.12** and a running MySQL 8.x with a `surveillance` database and user.

```
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

export MYSQL_HOST=127.0.0.1 MYSQL_PORT=3306
export MYSQL_DATABASE=surveillance MYSQL_USER=surveillance MYSQL_PASSWORD=surveillance

python manage.py migrate
python manage.py load_data
python manage.py runserver
```

Project layout

<code>config/</code>	Django project (settings, urls, wsgi)
<code>surveillance/</code>	main app
<code>models.py</code>	database schema
<code>management/commands/load_data.py</code>	the ETL
<code>analysis.py</code>	Support-Need Index + summary indicators
<code>views.py, urls.py</code>	dashboard / support / data-table pages
<code>templates/</code>	HTML + Chart.js
<code>tests/</code>	ETL, analysis and view tests
<code>etl/profile_data.py</code>	standalone data-profiling script
<code>data/raw/</code>	source CSV files
<code>docs/</code>	architecture diagram, data-quality report
<code>docker/</code>	container entrypoint

Data quality

Data quality is handled explicitly in the pipeline. The dataset is structurally clean (20 countries × 5 years, complete panels, no orphan country codes, no duplicate keys) but contains a number of value-level problems:

Issue	Where	Handling
Missing values	funding (6–7%), population under-5/urban (8%)	Kept as NULL — shown as "—", never imputed.
<code>deaths_reported > cases_reported / case_fatality_ratio_pct > 100%</code>	14 surveillance rows	Flagged as case under-ascertainment , kept and counted; CFR capped at 100% when averaging.
Noisy under-5 population	population	Not used in scoring; reported only.

The policy is **flag, never fabricate or drop**. Missing values stay NULL rather than being imputed. Rows where deaths exceed recorded cases (and the resulting CFR > 100%) are **not** treated as errors — in surveillance this usually reflects *case under-ascertainment* (a death recorded without the case being registered), so the rows are kept and counted. The CFR indicator uses the **median** (robust to the few >100% outliers, with no value modification). Crucially, under-ascertainment is **not just a note**: the share of a country's flagged records is itself an indicator in the index (weaker case-finding → higher need), so the flag affects the final assessment. Every flagged row is listed, with its reason, on the in-app **Data Quality** page. Run `python etl/profile_data.py` to regenerate `docs/data_quality_report.md`.

Methodology — Support-Need Index (SNI)

The SNI ranks countries by how much additional support they need. It uses ~14 indicators across five domains:

Domain	Weight	Indicators (oriented so higher = greater need)
Workforce	0.25	epidemiologists/100k, FETP-trained %, lab techs/100k
Reporting	0.20	timeliness %, completeness %, IDSR weekly compliance %, under-ascertainment rate
Laboratory	0.20	ISO 15189 accreditation %, turnaround days, tests/100k
Outbreaks	0.20	time-to-detection, outbreaks/year, median CFR
Funding	0.15	funding per capita, domestic funding share %

Steps:

1. Capacity indicators use the **latest year (2025)**; outbreak indicators use **5-year aggregates** (single-year outbreak counts are noisy).
2. Each indicator is **min-max normalised** across the 20 countries to a 0–1 "need" score, inverting direction where low values mean more need (e.g. low timeliness).
3. Indicators are averaged within a domain (missing values skipped), domains are combined with the weights above, and the result is scaled to **0–100**.
4. Countries are ranked and split into **High / Medium / Low need** by tertile.

Workforce carries the highest weight because field-epidemiology capacity is the structural bottleneck for every other function (IHR / Joint External Evaluation).

Why a transparent index and not machine learning

The index is intentionally transparent: each country's score decomposes into named indicators a manager can interrogate. With only 20 countries × 5 years there isn't enough data to train and validate a machine-learning model

without overfitting. Predictive modelling (e.g. outbreak-risk forecasting) is a sensible future extension once more history is available.

Recommendations (next biennium)

Derived from the weakest regional domains and the high-need cohort:

1. **Scale up the surveillance workforce (FETP)** — region averages **0.26 epidemiologists/100k**, far below the IHR target of 1.0. Prioritise the high-need cohort (Mali, DR Congo, CAR, Mozambique, Burkina Faso).
2. **Strengthen laboratory accreditation and turnaround** — only **~29%** of public labs are ISO 15189 accredited; average turnaround ~5.8 days.
3. **Improve reporting and reduce external-funding dependency** — timeliness ~75% and only ~39% of funding is domestic, leaving systems exposed to funding shocks.

Reproducibility

Reproducibility is treated as a first-class concern:

- **One-command setup** — `docker compose up --build` brings up MySQL and the app and loads the data, so any environment reproduces the same result.
- **Pinned base images** — both `python:3.12-slim` and `mysql:8.4` are pinned by `sha256` digest, so the build is identical regardless of when it runs.
- **Locked dependencies** — `requirements.txt` is generated from `requirements.in` with `pip-compile --generate-hashes`; every package (including transitive ones) is version-pinned and hash-verified.
- **Tests** — `python manage.py test surveillance` checks the ETL (correct counts and exactly 14 quarantined rows), the Support-Need Index (bounds and ranking) and that all pages return 200.
- **CI** — GitHub Actions runs migrations, the ETL and the tests against a fresh MySQL on every push, proving the project builds and runs on a clean machine.

To regenerate the lock file after changing `requirements.in`:

```
pip install pip-tools
pip-compile --generate-hashes --output-file=requirements.txt requirements.in
```

Limitations & possible extensions

- Synthetic data; indicator weights are an expert-judgement starting point and should be reviewed with the programme team (the code makes them a single dict to change).
- SNI uses min-max normalisation, so it is **relative** — it ranks within this cohort rather than against absolute international benchmarks.
- Extensions: configurable weights in the UI, a map view, authentication/roles for a real deployment, scheduled ingestion (Airflow/cron), and predictive outbreak-risk modelling once longer time series exist. ``